

# Chapter 16 — Linked List

## 0. What You Will Learn in This Chapter

Linked List is a foundational pointer-based pattern in coding interviews.

Unlike arrays, linked lists do not provide direct index access. You move from node to node using pointers.

Linked list problems test whether you can carefully manipulate:

- node pointers
- next links
- dummy nodes
- previous/current/next pointers
- head changes
- tail handling
- in-place reversal
- list merging
- deletion without losing nodes
- splitting and reconnecting lists
- recursive pointer thinking

By the end of this chapter, you should understand:

- What a linked list is
- Why linked lists exist
- How linked lists differ from arrays
- How to traverse a linked list
- How to use dummy nodes
- How to reverse a linked list
- How to merge linked lists
- How to remove nodes safely
- How to handle head deletion
- How to split and reconnect lists
- How to solve common linked list interview problems
- How to avoid pointer bugs
- How to explain linked list operations clearly

Core problems covered in this chapter:

1. Reverse Linked List
2. Merge Two Sorted Lists
3. Remove Linked List Elements
4. Remove Nth Node From End of List
5. Middle of the Linked List
6. Linked List Cycle
7. Palindrome Linked List
8. Reorder List
9. Swap Nodes in Pairs
10. Add Two Numbers
11. Odd Even Linked List
12. Partition List
13. Rotate List
14. Reverse Linked List II
15. Copy List With Random Pointer
16. Merge k Sorted Lists

17. Sort List
  18. Flatten a Multilevel Doubly Linked List
- 

# 1. What Is a Linked List?

A linked list is a sequence of nodes.

Each node stores:

19. a value
20. a pointer to the next node

Typical LeetCode definition:

```
struct ListNode {
    int val;
    ListNode* next;

    ListNode() : val(0), next(nullptr) {}

    ListNode(int x) : val(x), next(nullptr) {}

    ListNode(int x, ListNode* next) : val(x), next(next) {}
};
```

Example linked list:

1 -> 2 -> 3 -> 4 -> nullptr

Here:

- node 1 points to node 2
  - node 2 points to node 3
  - node 3 points to node 4
  - node 4 points to nullptr
- 

# 2. Why Linked Lists Exist

Linked lists are useful when nodes can be inserted or deleted without shifting elements.

In arrays, removing an element from the middle may require shifting many elements.

In linked lists, deletion is just pointer rewiring:

prev -> target -> next

After deletion:

prev -> next

No shifting required.

But linked lists have a cost:

- no  $O(1)$  random access
  - must traverse node by node
  - pointer manipulation is error-prone
-

### 3. Linked List vs Array

|                                       |                             |                           |
|---------------------------------------|-----------------------------|---------------------------|
|                                       |                             |                           |
| Access by index                       | $O(1)$                      | $O(n)$                    |
| Insert/delete at middle if node known | $O(n)$ shifting             | $O(1)$ pointer change     |
| Memory layout                         | contiguous                  | scattered nodes           |
| Traversal                             | simple index                | pointer movement          |
| Reverse                               | swap values or two pointers | pointer rewiring          |
| Common bugs                           | index out of bounds         | lost nodes / null pointer |

### 4. Core Need / Why Use Linked List Pattern?

Linked list problems require thinking in terms of **connections**, not indices.  
In arrays, you often write:

```
nums[i]  
nums[i + 1]
```

In linked lists, you write:

```
curr  
curr->next  
curr->next->next
```

You must carefully preserve nodes before changing links.

Example:

Wrong reversal:

```
curr->next = prev;  
curr = curr->next;
```

After `curr->next = prev`, the original next node is lost.

Correct:

```
ListNode* nextNode = curr->next;  
curr->next = prev;  
prev = curr;  
curr = nextNode;
```

# 5. When to Use Linked List Techniques

Use linked list techniques when the input is a linked list and the problem asks for:

## 5.1 Reversal

Examples:

- Reverse Linked List
- Reverse Linked List II
- Reverse Nodes in k-Group

## 5.2 Merging

Examples:

- Merge Two Sorted Lists
- Merge k Sorted Lists
- Sort List

## 5.3 Removal

Examples:

- Remove Linked List Elements
- Remove Nth Node From End
- Delete Node in a Linked List

## 5.4 Reordering

Examples:

- Reorder List
- Odd Even Linked List
- Partition List
- Rotate List

## 5.5 Cycle / Middle / Fast-Slow

Examples:

- Linked List Cycle
- Middle of the Linked List
- Palindrome Linked List

## 5.6 Copying Complex Pointers

Example:

- Copy List With Random Pointer
- 

# 6. When NOT to Use Linked List Pointer Tricks

## 6.1 If Converting to Array Is Allowed and Simpler

Some problems can be solved by copying values into a vector.

Example:

Palindrome Linked List can be solved by vector comparison.

But if  $O(1)$  space is requested, use linked list reversal.

## 6.2 If You Need Random Access

Linked lists do not support efficient index access.

If the problem needs frequent index lookup, use array/vector if possible.

### 6.3 If Original Nodes Must Not Be Modified

Many pointer tricks mutate the list.

If modification is not allowed, copy values or create new nodes.

---

## 7. Core Intuition

Linked list problems usually ask:

Which pointer should point where next?

At each step, think about:

21. current node
22. previous node
23. next node
24. whether head changes
25. whether tail changes
26. whether any node is lost

The most important rule:

Save the next pointer before changing curr->next.

---

## 8. Mental Models

### 8.1 Train Cars

Each node is a train car connected to the next car.

Changing links is like disconnecting and reconnecting train cars.

### 8.2 Pointer Surgery

You are performing surgery on links.

Before cutting a link, know where both ends should reconnect.

### 8.3 Dummy Node as Fake Head

A dummy node sits before the real head.

It simplifies deletion and insertion at the beginning.

dummy -> head -> ...

If head changes, dummy still points to the correct new list.

### 8.4 Three-Pointer Reversal

For reversal, remember:

prev, curr, next

You need all three.

---

## 9. C++ Tools Needed

### 9.1 Node Pointer

```
ListNode* curr = head;
```

### 9.2 Null Check

Always check before dereferencing:

```
if (curr != nullptr) {  
    curr = curr->next;  
}
```

Wrong:

```
curr->next
```

when curr could be nullptr.

### 9.3 Dummy Node

```
ListNode dummy(0);  
dummy.next = head;
```

```
return dummy.next;
```

### 9.4 Dynamic Dummy Node

Usually avoid unless needed:

```
ListNode* dummy = new ListNode(0);
```

For most LeetCode solutions, stack-allocated dummy is enough:

```
ListNode dummy(0);
```

## 10. Core Templates

### Template 1 — Traverse Linked List

```
ListNode* curr = head;
```

```
while (curr != nullptr) {  
    // process curr->val  
    curr = curr->next;  
}
```

### Template 2 — Reverse Linked List

```
ListNode* reverseList(ListNode* head) {  
    ListNode* prev = nullptr;  
    ListNode* curr = head;
```

```
    while (curr != nullptr) {  
        ListNode* nextNode = curr->next;
```

```
        curr->next = prev;
```

```
        prev = curr;
```

```
        curr = nextNode;
    }

    return prev;
}
```

### Template 3 — Dummy Node for Deletion

```
ListNode dummy(0);
dummy.next = head;

ListNode* curr = &dummy;

while (curr->next != nullptr) {
    if (curr->next->val == target) {
        curr->next = curr->next->next;
    } else {
        curr = curr->next;
    }
}

return dummy.next;
```

### Template 4 — Merge Two Sorted Lists

```
ListNode dummy(0);
ListNode* tail = &dummy;

while (l1 != nullptr && l2 != nullptr) {
    if (l1->val <= l2->val) {
        tail->next = l1;
        l1 = l1->next;
    } else {
        tail->next = l2;
        l2 = l2->next;
    }

    tail = tail->next;
}

tail->next = l1 != nullptr ? l1 : l2;

return dummy.next;
```

---

## Template 5 — Fast and Slow Pointer

```
ListNode* slow = head;
ListNode* fast = head;

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
```

Used for:

- middle node
- cycle detection
- splitting list

---

## Template 6 — Find Previous Node Before Target Position

```
ListNode dummy(0);
dummy.next = head;

ListNode* prev = &dummy;

for (int i = 0; i < position; i++) {
    prev = prev->next;
}
```

Then `prev->next` is the node to operate on.

---

# 11. Major Variations of Linked List Problems

| Traversal           | move curr         | Count nodes            |
|---------------------|-------------------|------------------------|
| Reversal            | prev/curr/next    | Reverse Linked List    |
| Dummy node deletion | fake head         | Remove Elements        |
| Merge               | tail pointer      | Merge Two Sorted Lists |
| Fast-slow           | two speeds        | Middle / Cycle         |
| Two pointers gap    | maintain distance | Remove Nth From End    |
| Split and reverse   | combine patterns  | Palindrome / Reorder   |

| Variation          | Core Idea              | Example         |
|--------------------|------------------------|-----------------|
| Reconnect groups   | local pointer surgery  | Swap Pairs      |
| Carry simulation   | digit addition         | Add Two Numbers |
| Hash map copy      | clone complex pointers | Random Pointer  |
| Divide and conquer | split/merge            | Sort List       |

## 12. Full Problem Walkthrough 1 — LeetCode 206: Reverse Linked List

### 12.1 Problem Summary

Given head of linked list, reverse it.

Example:

1 -> 2 -> 3 -> 4 -> 5

Output:

5 -> 4 -> 3 -> 2 -> 1

### 12.2 How to Identify This Pattern

Problem directly asks to reverse a linked list.

Use three pointers:

prev, curr, nextNode

### 12.3 Brute Force Idea

Copy values into vector, then rewrite them reversed.

This works but does not reverse node links and uses  $O(n)$  extra space.

### 12.4 Optimized Intuition

At each node:

27. save next node
28. reverse current link
29. move prev forward
30. move curr forward

### 12.5 C++ Code

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        ListNode* prev = nullptr;
        ListNode* curr = head;
```

```

while (curr != nullptr) {
    ListNode* nextNode = curr->next;

    curr->next = prev;

    prev = curr;
    curr = nextNode;
}

return prev;
}
};

```

## 12.6 Dry Run

Input:

1 -> 2 -> 3 -> nullptr

Initial:

prev = nullptr  
curr = 1

| 1 | 2    | 1 -> nullptr | 1 | 2    |
|---|------|--------------|---|------|
| 2 | 3    | 2 -> 1       | 2 | 3    |
| 3 | null | 3 -> 2       | 3 | null |

Return prev.

Output:

3 -> 2 -> 1 -> nullptr

## 12.7 Correctness Explanation

Each iteration reverses one link and moves the processed node into the reversed prefix. prev always points to the head of the reversed part, and curr points to the remaining unreversed part.

## 12.8 Derived Complexity Proof

Each node is visited once.

Each node's next pointer is changed once.

Time:

$O(n)$

Space:

$O(1)$

## 12.9 Edge Cases

- empty list
- one node
- two nodes
- long list

## 12.10 Common Mistakes

### Mistake 1 — Losing next node

Wrong:

```
curr->next = prev;  
curr = curr->next;
```

Correct:

```
ListNode* nextNode = curr->next;  
curr->next = prev;  
curr = nextNode;
```

### Mistake 2 — Returning head

After reversal, old head becomes tail.

Return prev.

## 12.11 Similar Problems

- Reverse Linked List II
- Reverse Nodes in k-Group
- Palindrome Linked List
- Reorder List

# 13. Full Problem Walkthrough 2 — LeetCode 21: Merge Two Sorted Lists

## 13.1 Problem Summary

Given two sorted linked lists, merge them into one sorted list.

Example:

$l1 = 1 \rightarrow 2 \rightarrow 4$

$l2 = 1 \rightarrow 3 \rightarrow 4$

Output:

$1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 4$

---

## 13.2 How to Identify This Pattern

Problem asks to merge two sorted lists.

This is same as merge step of merge sort.

Use dummy node and tail pointer.

---

## 13.3 C++ Code

```
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        ListNode dummy(0);
        ListNode* tail = &dummy;

        while (list1 != nullptr && list2 != nullptr) {
            if (list1->val <= list2->val) {
                tail->next = list1;
                list1 = list1->next;
            } else {
                tail->next = list2;
                list2 = list2->next;
            }

            tail = tail->next;
        }

        if (list1 != nullptr) {
            tail->next = list1;
        } else {
            tail->next = list2;
        }

        return dummy.next;
    }
};
```

## 13.4 Dry Run

l1 = 1 -> 2 -> 4

l2 = 1 -> 3 -> 4

|        |    |   |
|--------|----|---|
|        |    |   |
| 1 vs 1 | l1 | 1 |

| compare  | choose    | merged                     |
|----------|-----------|----------------------------|
| 2 vs 1   | l2        | 1 -> 1                     |
| 2 vs 3   | l1        | 1 -> 1 -> 2                |
| 4 vs 3   | l2        | 1 -> 1 -> 2 -> 3           |
| 4 vs 4   | l1        | 1 -> 1 -> 2 -> 3 -> 4      |
| l1 empty | append l2 | 1 -> 1 -> 2 -> 3 -> 4 -> 4 |

## 13.5 Complexity

Time:

$O(n + m)$

Space:

$O(1)$

Reuses existing nodes.

## 13.6 Common Mistakes

- Not moving tail
- Not appending remaining list
- Creating unnecessary new nodes
- Losing list pointer before advancing

# 14. Full Problem Walkthrough 3 — LeetCode 203: Remove Linked List Elements

## 14.1 Problem Summary

Remove all nodes with value val.

Example:

head = 1 -> 2 -> 6 -> 3 -> 4 -> 5 -> 6

val = 6

Output:

1 -> 2 -> 3 -> 4 -> 5

## 14.2 How to Identify This Pattern

Need remove nodes, possibly including head.

Use dummy node.

## 14.3 C++ Code

```
class Solution {
public:
    ListNode* removeElements(ListNode* head, int val) {
        ListNode dummy(0);
        dummy.next = head;

        ListNode* curr = &dummy;

        while (curr->next != nullptr) {
            if (curr->next->val == val) {
                curr->next = curr->next->next;
            } else {
                curr = curr->next;
            }
        }

        return dummy.next;
    }
};
```

## 14.4 Why Dummy Node?

If head itself must be removed, dummy handles it naturally.

Example:

6 -> 6 -> 1

Dummy:

dummy -> 6 -> 6 -> 1

Remove nodes by updating dummy.next.

## 14.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

## 14.6 Common Mistake

After deletion, do not advance curr.

Wrong:

```
curr->next = curr->next->next;  
curr = curr->next;
```

If multiple consecutive nodes must be deleted, this skips checking one.  
Correct:

```
curr->next = curr->next->next;
```

Only advance when not deleting.

## 15. Full Problem Walkthrough 4 — LeetCode 19: Remove Nth Node From End of List

### 15.1 Problem Summary

Remove the nth node from the end of list.

Example:

```
head = 1 -> 2 -> 3 -> 4 -> 5  
n = 2
```

Output:

```
1 -> 2 -> 3 -> 5
```

### 15.2 How to Identify This Pattern

Need remove node based on distance from end.

Use two pointers with gap n.

### 15.3 Intuition

Move fast ahead by n steps.

Then move both fast and slow until fast reaches end.

slow->next is the node to delete.

Use dummy to handle deleting head.

### 15.4 C++ Code

```
class Solution {  
public:  
    ListNode* removeNthFromEnd(ListNode* head, int n) {  
        ListNode dummy(0);  
        dummy.next = head;  
  
        ListNode* fast = &dummy;  
        ListNode* slow = &dummy;
```

```
    for (int i = 0; i < n; i++) {
        fast = fast->next;
    }

    while (fast->next != nullptr) {
        fast = fast->next;
        slow = slow->next;
    }

    slow->next = slow->next->next;

    return dummy.next;
}
};
```

## 15.5 Dry Run

List:

1 -> 2 -> 3 -> 4 -> 5  
n = 2

Move fast 2 ahead.  
Then move both until fast is at last node.  
slow stops at node 3.  
Delete slow->next, which is 4.

## 15.6 Complexity

Time:

$O(n)$

Space:

$O(1)$

## 15.7 Edge Cases

- remove head
- remove tail
- one-node list
- n equals list length

## 15.8 Common Mistakes

- Not using dummy and failing when deleting head
- Moving fast too many or too few steps
- Using while (fast != nullptr) with wrong gap

---

# 16. Full Problem Walkthrough 5 — LeetCode 876: Middle of the Linked List

## 16.1 Problem Summary

Return middle node.

If there are two middle nodes, return the second one.

Example:

1 -> 2 -> 3 -> 4 -> 5

Output:

3

Example:

1 -> 2 -> 3 -> 4 -> 5 -> 6

Output:

4

## 16.2 How to Identify This Pattern

Need middle.

Use fast and slow pointers.

Fast moves two steps.

Slow moves one step.

When fast reaches end, slow is middle.

---

## 16.3 C++ Code

```
class Solution {
public:
    ListNode* middleNode(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }

        return slow;
    }
};
```

---

## 16.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

## 16.5 Edge Cases

- one node
  - two nodes
  - odd length
  - even length
- 

# 17. Full Problem Walkthrough 6 — LeetCode 141: Linked List Cycle

## 17.1 Problem Summary

Return true if linked list has a cycle.

---

## 17.2 How to Identify This Pattern

Cycle detection in linked list uses Floyd's algorithm.

Use:

- slow pointer moves one step
- fast pointer moves two steps

If cycle exists, fast eventually meets slow.

---

## 17.3 C++ Code

```
class Solution {
public:
    bool hasCycle(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;

            if (slow == fast) {
                return true;
            }
        }
    }
}
```

```
        return false;
    }
};
```

## 17.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

## 17.5 Common Mistakes

- Checking fast->next without checking fast
- Comparing node values instead of node pointers
- Using hash set when  $O(1)$  space is required

# 18. Full Problem Walkthrough 7 — LeetCode 234: Palindrome Linked List

## 18.1 Problem Summary

Return true if linked list is palindrome.

Example:

1 -> 2 -> 2 -> 1

Output:

true

## 18.2 How to Identify This Pattern

Need compare first half and second half.

For  $O(1)$  space:

31. find middle
32. reverse second half
33. compare halves

## 18.3 C++ Code

```
class Solution {
public:
    ListNode* reverse(ListNode* head) {
        ListNode* prev = nullptr;
```

```

ListNode* curr = head;

while (curr != nullptr) {
    ListNode* nextNode = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nextNode;
}

return prev;
}

bool isPalindrome(ListNode* head) {
    if (head == nullptr || head->next == nullptr) {
        return true;
    }

    ListNode* slow = head;
    ListNode* fast = head;

    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    ListNode* second = reverse(slow);
    ListNode* first = head;

    while (second != nullptr) {
        if (first->val != second->val) {
            return false;
        }

        first = first->next;
        second = second->next;
    }

    return true;
}
};

```

## 18.4 Complexity

Time:

$O(n)$

Space:

$O(1)$

## 18.5 Common Mistakes

- Not reversing second half
- Comparing wrong number of nodes
- Mishandling odd length
- Forgetting list is modified, if restoration is required

# 19. Full Problem Walkthrough 8 — LeetCode 143: Reorder List

## 19.1 Problem Summary

Given list:

$L_0 \rightarrow L_1 \rightarrow L_2 \rightarrow \dots \rightarrow L_n$

Reorder to:

$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \dots$

Example:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

Output:

$1 \rightarrow 4 \rightarrow 2 \rightarrow 3$

## 19.2 How to Identify This Pattern

Need combine front half and reversed back half.

Steps:

34. find middle
35. split list
36. reverse second half
37. merge alternating

## 19.3 C++ Code

```
class Solution {
public:
    ListNode* reverse(ListNode* head) {
        ListNode* prev = nullptr;
        ListNode* curr = head;
```

```

while (curr != nullptr) {
    ListNode* nextNode = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nextNode;
}

return prev;
}

void reorderList(ListNode* head) {
    if (head == nullptr || head->next == nullptr) {
        return;
    }

    ListNode* slow = head;
    ListNode* fast = head;

    while (fast->next != nullptr && fast->next->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
    }

    ListNode* second = slow->next;
    slow->next = nullptr;

    second = reverse(second);

    ListNode* first = head;

    while (second != nullptr) {
        ListNode* temp1 = first->next;
        ListNode* temp2 = second->next;

        first->next = second;
        second->next = temp1;

        first = temp1;
        second = temp2;
    }
}
};

```

---

## 19.4 Dry Run

Input:

1 -> 2 -> 3 -> 4 -> 5

Split:

1 -> 2 -> 3

4 -> 5

Reverse second:

5 -> 4

Merge:

1 -> 5 -> 2 -> 4 -> 3

## 19.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

## 19.6 Common Mistakes

- Not splitting list before reversing
  - Creating a cycle accidentally
  - Losing next pointers during merge
  - Wrong middle for even length
- 

# 20. Full Problem Walkthrough 9 — LeetCode 24: Swap Nodes in Pairs

## 20.1 Problem Summary

Swap every two adjacent nodes.

Example:

1 -> 2 -> 3 -> 4

Output:

2 -> 1 -> 4 -> 3

## 20.2 How to Identify This Pattern

Need reconnect nodes in pairs.

Use dummy node and pointer surgery.

---

## 20.3 C++ Code

```
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode dummy(0);
        dummy.next = head;

        ListNode* prev = &dummy;

        while (prev->next != nullptr && prev->next->next != nullptr) {
            ListNode* first = prev->next;
            ListNode* second = first->next;

            first->next = second->next;
            second->next = first;
            prev->next = second;

            prev = first;
        }

        return dummy.next;
    }
};
```

## 20.4 Pointer Rewiring

Before:

prev -> first -> second -> next

After:

prev -> second -> first -> next

Steps:

```
first->next = second->next;
second->next = first;
prev->next = second;
```

## 20.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

---

# 21. Full Problem Walkthrough 10 — LeetCode 2: Add Two Numbers

## 21.1 Problem Summary

Two numbers are represented in reverse order as linked lists.  
Add them and return sum as linked list.

Example:

l1 = 2 -> 4 -> 3  
l2 = 5 -> 6 -> 4

Represents:

342 + 465 = 807

Output:

7 -> 0 -> 8

## 21.2 How to Identify This Pattern

This is digit-by-digit addition with carry.  
Use dummy node and tail pointer.

---

## 21.3 C++ Code

```
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode dummy(0);
        ListNode* tail = &dummy;

        int carry = 0;

        while (l1 != nullptr || l2 != nullptr || carry != 0) {
            int sum = carry;

            if (l1 != nullptr) {
                sum += l1->val;
                l1 = l1->next;
            }

            if (l2 != nullptr) {
                sum += l2->val;
                l2 = l2->next;
            }

            int val = sum % 10;
            sum = sum / 10;

            tail->next = new ListNode(val);
            tail = tail->next;
        }

        return dummy.next;
    }
};
```

```

        carry = sum / 10;
        int digit = sum % 10;

        tail->next = new ListNode(digit);
        tail = tail->next;
    }

    return dummy.next;
}
};

```

## 21.4 Complexity

Time:

$O(\max(n, m))$

Space:

$O(\max(n, m))$

Output list space.

## 21.5 Common Mistakes

- Forgetting final carry
- Mishandling unequal lengths
- Trying to convert entire list to integer, causing overflow

# 22. Full Problem Walkthrough 11 — LeetCode 328: Odd Even Linked List

## 22.1 Problem Summary

Group all nodes at odd indices followed by nodes at even indices.

Indices are 1-based.

Example:

1 -> 2 -> 3 -> 4 -> 5

Output:

1 -> 3 -> 5 -> 2 -> 4

## 22.2 How to Identify This Pattern

Need separate nodes into two chains while preserving relative order.

Use odd and even pointers.

---

## 22.3 C++ Code

```
class Solution {
public:
    ListNode* oddEvenList(ListNode* head) {
        if (head == nullptr) {
            return nullptr;
        }

        ListNode* odd = head;
        ListNode* even = head->next;
        ListNode* evenHead = even;

        while (even != nullptr && even->next != nullptr) {
            odd->next = even->next;
            odd = odd->next;

            even->next = odd->next;
            even = even->next;
        }

        odd->next = evenHead;

        return head;
    }
};
```

## 22.4 Complexity

Time:

|        |
|--------|
| $O(n)$ |
|--------|

Space:

|        |
|--------|
| $O(1)$ |
|--------|

# 23. Full Problem Walkthrough 12 — LeetCode 86: Partition List

## 23.1 Problem Summary

Given list and value  $x$ , partition list so nodes less than  $x$  come before nodes greater than or equal to  $x$ .

Preserve original relative order.

Example:

1 -> 4 -> 3 -> 2 -> 5 -> 2  
x = 3

Output:

1 -> 2 -> 2 -> 4 -> 3 -> 5

## 23.2 How to Identify This Pattern

Need stable partition into two lists.

Use two dummy nodes:

- smaller list
- greater/equal list

## 23.3 C++ Code

```
class Solution {
public:
    ListNode* partition(ListNode* head, int x) {
        ListNode beforeDummy(0);
        ListNode afterDummy(0);

        ListNode* before = &beforeDummy;
        ListNode* after = &afterDummy;

        while (head != nullptr) {
            if (head->val < x) {
                before->next = head;
                before = before->next;
            } else {
                after->next = head;
                after = after->next;
            }

            head = head->next;
        }

        after->next = nullptr;
        before->next = afterDummy.next;

        return beforeDummy.next;
    }
};
```

## 23.4 Important Detail

Set:

after->next = nullptr;

Otherwise old links may create incorrect chains or cycles.

## 23.5 Complexity

Time:

$O(n)$

Space:

$O(1)$

# 24. Full Problem Walkthrough 13 — LeetCode 61: Rotate List

## 24.1 Problem Summary

Rotate list to the right by  $k$  places.

Example:

1 -> 2 -> 3 -> 4 -> 5

$k = 2$

Output:

4 -> 5 -> 1 -> 2 -> 3

## 24.2 How to Identify This Pattern

Need move last  $k$  nodes to front.

Find length, connect tail to head, then break at new tail.

## 24.3 C++ Code

```
class Solution {
public:
    ListNode* rotateRight(ListNode* head, int k) {
        if (head == nullptr || head->next == nullptr || k == 0) {
            return head;
        }

        int length = 1;
        ListNode* tail = head;

        while (tail->next != nullptr) {
```

```

        tail = tail->next;
        length++;
    }

    k %= length;

    if (k == 0) {
        return head;
    }

    tail->next = head;

    int stepsToNewTail = length - k - 1;
    ListNode* newTail = head;

    for (int i = 0; i < stepsToNewTail; i++) {
        newTail = newTail->next;
    }

    ListNode* newHead = newTail->next;
    newTail->next = nullptr;

    return newHead;
}
};

```

## 24.4 Complexity

Time:

O(n)

Space:

O(1)

## 24.5 Common Mistakes

- Forgetting `k %= length`
- Not breaking cycle
- Mishandling empty list
- Off-by-one in new tail position

# 25. Full Problem Walkthrough 14 — LeetCode 92: Reverse Linked List II

## 25.1 Problem Summary

Reverse nodes from position left to right.

Positions are 1-indexed.

Example:

```
1 -> 2 -> 3 -> 4 -> 5
```

```
left = 2
```

```
right = 4
```

Output:

```
1 -> 4 -> 3 -> 2 -> 5
```

## 25.2 How to Identify This Pattern

Need reverse only a sublist.

Use dummy node to reach node before reversal.

Then reverse by front-insertion.

## 25.3 C++ Code

```
class Solution {
public:
    ListNode* reverseBetween(ListNode* head, int left, int right) {
        if (head == nullptr || left == right) {
            return head;
        }

        ListNode dummy(0);
        dummy.next = head;

        ListNode* prev = &dummy;

        for (int i = 1; i < left; i++) {
            prev = prev->next;
        }

        ListNode* curr = prev->next;

        for (int i = 0; i < right - left; i++) {
            ListNode* move = curr->next;
```

```
        curr->next = move->next;
        move->next = prev->next;
        prev->next = move;
    }

    return dummy.next;
}
```

## 25.4 Dry Run

Input:

```
1 -> 2 -> 3 -> 4 -> 5
left = 2
right = 4
```

prev points to 1.

curr points to 2.

Move 3 after prev:

```
1 -> 3 -> 2 -> 4 -> 5
```

Move 4 after prev:

```
1 -> 4 -> 3 -> 2 -> 5
```

## 25.5 Complexity

Time:

```
O(n)
```

Space:

```
O(1)
```

## 25.6 Common Mistakes

- Losing move->next
- Not using dummy when left = 1
- Reversing too many nodes
- Returning old head incorrectly

# 26. Full Problem Walkthrough 15 — LeetCode 138: Copy List With Random Pointer

## 26.1 Problem Summary

Each node has:

next  
random

random can point to any node or nullptr.

Return a deep copy.

## 26.2 How to Identify This Pattern

Need clone nodes while preserving arbitrary random links.

Use hash map:

old node -> new node

## 26.3 C++ Code

```
class Solution {
public:
    Node* copyRandomList(Node* head) {
        if (head == nullptr) {
            return nullptr;
        }

        unordered_map<Node*, Node*> mp;

        Node* curr = head;

        while (curr != nullptr) {
            mp[curr] = new Node(curr->val);
            curr = curr->next;
        }

        curr = head;

        while (curr != nullptr) {
            mp[curr]->next = curr->next == nullptr ? nullptr : mp[curr->next];
            mp[curr]->random = curr->random == nullptr ? nullptr :
mp[curr->random];

            curr = curr->next;
        }

        return mp[head];
    }
};
```

```
}  
};
```

## 26.4 Complexity

Time:

$O(n)$

Space:

$O(n)$

## 26.5 Common Mistakes

- Shallow copying random pointers
- Creating clone only when first visited by next
- Not handling random = nullptr
- Returning original head

# 27. Full Problem Walkthrough 16 — LeetCode 23: Merge k Sorted Lists

## 27.1 Problem Summary

Merge k sorted linked lists into one sorted list.

## 27.2 How to Identify This Pattern

This is merging multiple sorted streams.

Use min heap.

## 27.3 C++ Code

```
class Solution {  
public:  
    struct Compare {  
        bool operator()(ListNode* a, ListNode* b) {  
            return a->val > b->val;  
        }  
    };  
  
    ListNode* mergeKLists(vector<ListNode*>& lists) {  
        priority_queue<ListNode*, vector<ListNode*>, Compare> pq;  
  
        for (ListNode* node : lists) {  
            if (node != nullptr) {  
                pq.push(node);  
            }  
        }  
    }  
};
```

```

    }
}

ListNode dummy(0);
ListNode* tail = &dummy;

while (!pq.empty()) {
    ListNode* node = pq.top();
    pq.pop();

    tail->next = node;
    tail = tail->next;

    if (node->next != nullptr) {
        pq.push(node->next);
    }
}

return dummy.next;
}
};

```

## 27.4 Complexity

Let  $N$  be total nodes and  $k$  be number of lists.

Time:

$O(N \log k)$

Space:

$O(k)$

# 28. Full Problem Walkthrough 17 — LeetCode 148: Sort List

## 28.1 Problem Summary

Sort a linked list in  $O(n \log n)$  time and  $O(1)$  extra space.

Example:

4 -> 2 -> 1 -> 3

Output:

1 -> 2 -> 3 -> 4

## 28.2 How to Identify This Pattern

Linked list sorting is best done with merge sort.

Why?

- splitting list is  $O(n)$
  - merging lists is natural
  - no random access needed
- 

## 28.3 C++ Code

```
class Solution {
public:
    ListNode* merge(ListNode* a, ListNode* b) {
        ListNode dummy(0);
        ListNode* tail = &dummy;

        while (a != nullptr && b != nullptr) {
            if (a->val <= b->val) {
                tail->next = a;
                a = a->next;
            } else {
                tail->next = b;
                b = b->next;
            }

            tail = tail->next;
        }

        tail->next = a != nullptr ? a : b;

        return dummy.next;
    }

    ListNode* sortList(ListNode* head) {
        if (head == nullptr || head->next == nullptr) {
            return head;
        }

        ListNode* slow = head;
        ListNode* fast = head->next;

        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }
    }
};
```

```

    }

    ListNode* second = slow->next;
    slow->next = nullptr;

    ListNode* left = sortList(head);
    ListNode* right = sortList(second);

    return merge(left, right);
}
};

```

## 28.4 Complexity

Time:

$O(n \log n)$

Space:

$O(\log n)$

due to recursion stack.

## 28.5 Common Mistakes

- Not splitting list with `slow->next = nullptr`
- Infinite recursion
- Losing second half
- Trying quicksort on linked list, which is usually less natural

# 29. Mini Problem 1 — Count Nodes in Linked List

```

int countNodes(ListNode* head) {
    int count = 0;

    while (head != nullptr) {
        count++;
        head = head->next;
    }

    return count;
}

```

## 30. Mini Problem 2 — Delete Node When Only Given That Node

LeetCode 237.

```
class Solution {
public:
    void deleteNode(ListNode* node) {
        node->val = node->next->val;
        node->next = node->next->next;
    }
};
```

Works because node is not tail.

## 31. Mini Problem 3 — Intersection of Two Linked Lists

Use length difference or two-pointer switching.

```
class Solution {
public:
    ListNode* getIntersectionNode(ListNode* headA, ListNode* headB) {
        ListNode* a = headA;
        ListNode* b = headB;

        while (a != b) {
            a = a == nullptr ? headB : a->next;
            b = b == nullptr ? headA : b->next;
        }

        return a;
    }
};
```

If no intersection, both become nullptr.

## 32. Mini Problem 4 — Reverse First K Nodes

```
ListNode* reverseK(ListNode* head, int k) {
    ListNode* prev = nullptr;
```

```
ListNode* curr = head;

while (k-- && curr != nullptr) {
    ListNode* nextNode = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nextNode;
}

return prev;
}
```

Used inside Reverse Nodes in k-Group.

## 33. Common Edge Cases for Linked List

### 33.1 Empty List

Always handle:

```
head == nullptr
```

### 33.2 One Node

Many pointer rewiring loops should not run.

### 33.3 Two Nodes

Good test for reversal, swapping, cycle, middle.

### 33.4 Head Changes

Use dummy node when head might be removed or changed.

### 33.5 Tail Changes

Remember to set tail to nullptr when splitting or partitioning.

### 33.6 Cycle Creation

Wrong pointer rewiring may create cycle.

Example:

```
tail->next = oldNode;
```

without breaking old links.

### 33.7 Losing Nodes

Save next pointer before rewiring.

## 34. Common Mistakes and Wrong Code

### Mistake 1 — Losing the Rest of the List

Wrong:

```
curr->next = prev;  
curr = curr->next;
```

Correct:

```
ListNode* nextNode = curr->next;  
curr->next = prev;  
curr = nextNode;
```

## Mistake 2 — Not Using Dummy Node

Problems where head can change become harder.

Use dummy for deletion, insertion, and partial reversal.

---

## Mistake 3 — Dereferencing Null

Wrong:

```
while (fast->next != nullptr)
```

if fast may be null.

Correct:

```
while (fast != nullptr && fast->next != nullptr)
```

## Mistake 4 — Forgetting to Move Pointer

Can cause infinite loop.

Example:

```
while (curr != nullptr) {  
    // forgot curr = curr->next  
}
```

## Mistake 5 — Creating Cycles Accidentally

After partitioning or splitting, ensure final tail points to nullptr.

---

# 35. Pattern Recognition Checklist

Ask:

- 38. Can head be removed or changed?
- 39. Should I use dummy node?
- 40. Am I reversing links?
- 41. Do I need fast and slow pointers?
- 42. Do I need to remove a node?
- 43. Do I need previous node?
- 44. Do I need to preserve next node before rewiring?
- 45. Am I merging sorted lists?
- 46. Am I splitting list into halves?
- 47. Could my pointer changes create a cycle?

---

## 36. Interview Explanation Templates

### 36.1 Reverse Linked List

I iterate through the list and reverse one pointer at a time. Before changing `curr->next`, I save the original next node. Then I redirect `curr->next` to `prev` and move both pointers forward.

### 36.2 Dummy Node

I use a dummy node before head so deletion or insertion near the head is handled the same way as any other node. This avoids special-casing head changes.

### 36.3 Merge Two Lists

I use a dummy head and a tail pointer. At each step, I attach the smaller current node from the two lists and move that list pointer forward.

### 36.4 Remove Nth From End

I use two pointers with a gap of  $n$ . When the fast pointer reaches the end, the slow pointer is right before the node to remove.

### 36.5 Reorder List

I split the list into two halves, reverse the second half, then merge the two halves alternately.

---

## 37. Complexity Patterns

| Traverse list          | $O(n)$          | $O(1)$             |
|------------------------|-----------------|--------------------|
| Reverse list           | $O(n)$          | $O(1)$             |
| Merge two sorted lists | $O(n + m)$      | $O(1)$             |
| Remove elements        | $O(n)$          | $O(1)$             |
| Remove nth from end    | $O(n)$          | $O(1)$             |
| Middle node            | $O(n)$          | $O(1)$             |
| Cycle detection        | $O(n)$          | $O(1)$             |
| Palindrome list        | $O(n)$          | $O(1)$             |
| Reorder list           | $O(n)$          | $O(1)$             |
| Add two numbers        | $O(\max(n, m))$ | $O(\text{output})$ |
| Copy random pointer    | $O(n)$          | $O(n)$             |

| Problem Type  | Time          | Space       |
|---------------|---------------|-------------|
| Merge k lists | $O(N \log k)$ | $O(k)$      |
| Sort list     | $O(n \log n)$ | $O(\log n)$ |

## 38. Time-Boxed Practice Guidance

### Easy Linked List Problems

Target:

10–20 minutes

Examples:

- Reverse Linked List
- Merge Two Sorted Lists
- Remove Linked List Elements
- Middle of the Linked List
- Linked List Cycle

### Medium Linked List Problems

Target:

25–45 minutes

Examples:

- Remove Nth Node From End
- Add Two Numbers
- Odd Even Linked List
- Partition List
- Swap Nodes in Pairs
- Palindrome Linked List

### Harder Linked List Problems

Target:

45–75 minutes

Examples:

- Reorder List
- Reverse Linked List II
- Copy List With Random Pointer
- Merge k Sorted Lists
- Sort List
- Reverse Nodes in k-Group

## 39. Expanded Practice Set

### 39.1 Must-Do Easy

48. Reverse Linked List
49. Merge Two Sorted Lists
50. Remove Linked List Elements

- 51. Middle of the Linked List
- 52. Linked List Cycle
- 53. Intersection of Two Linked Lists
- 54. Delete Node in a Linked List

### 39.2 Must-Do Medium

- 55. Remove Nth Node From End of List
- 56. Add Two Numbers
- 57. Odd Even Linked List
- 58. Partition List
- 59. Swap Nodes in Pairs
- 60. Palindrome Linked List
- 61. Reorder List
- 62. Rotate List
- 63. Reverse Linked List II
- 64. Copy List With Random Pointer

### 39.3 Strong Interview Practice

- 65. Merge k Sorted Lists
- 66. Sort List
- 67. Reverse Nodes in k-Group
- 68. Linked List Cycle II
- 69. Flatten a Multilevel Doubly Linked List
- 70. Add Two Numbers II
- 71. Design Linked List
- 72. LRU Cache, linked list + hash map

### 39.4 Hard / Stretch

- 73. Reverse Nodes in k-Group
  - 74. Merge k Sorted Lists
  - 75. Sort List
  - 76. LRU Cache
  - 77. LFU Cache
  - 78. Flatten a Multilevel Doubly Linked List
- 

## 40. Suggested Study Order

### Step 1 — Basic Traversal and Reversal

Start with:

- 79. Reverse Linked List
- 80. Middle of the Linked List
- 81. Remove Linked List Elements

Goal:

Become comfortable with curr, next, and dummy node.

---

### Step 2 — Merge and Delete

Solve:

- 82. Merge Two Sorted Lists
- 83. Remove Nth Node From End
- 84. Delete Node in a Linked List

Goal:

Master dummy and tail pointers.

---

### Step 3 — Fast/Slow Pointer Problems

Solve:

- 85. Linked List Cycle
- 86. Linked List Cycle II
- 87. Palindrome Linked List

Goal:

Master fast-slow movement.

---

### Step 4 — Reordering and Partitioning

Solve:

- 88. Reorder List
- 89. Odd Even Linked List
- 90. Partition List
- 91. Rotate List

Goal:

Learn splitting and reconnecting.

---

### Step 5 — Advanced Pointer Surgery

Solve:

- 92. Swap Nodes in Pairs
- 93. Reverse Linked List II
- 94. Reverse Nodes in k-Group

Goal:

Handle local reversals safely.

---

### Step 6 — Complex Structures

Solve:

- 95. Copy List With Random Pointer
- 96. Merge k Sorted Lists
- 97. Sort List

Goal:

Combine linked list with hash map, heap, and merge sort.

---

## 41. Pattern Decision Table

| reverse entire list | prev/curr/next   | Reverse Linked List |
|---------------------|------------------|---------------------|
| remove nodes        | dummy node       | Remove Elements     |
| merge sorted lists  | dummy + tail     | Merge Two Lists     |
| nth from end        | two pointers gap | Remove Nth Node     |
| middle node         | fast/slow        | Middle of List      |

| Problem Clue        | Use This Technique           | Example             |
|---------------------|------------------------------|---------------------|
| cycle exists        | Floyd fast/slow              | Linked List Cycle   |
| palindrome          | middle + reverse second half | Palindrome List     |
| reorder front/back  | split + reverse + merge      | Reorder List        |
| swap adjacent nodes | dummy + pointer surgery      | Swap Pairs          |
| add digits          | carry + dummy                | Add Two Numbers     |
| stable partition    | two dummy lists              | Partition List      |
| rotate              | length + circular link       | Rotate List         |
| copy random         | hash map old to new          | Copy Random Pointer |
| sort linked list    | merge sort                   | Sort List           |
| merge many lists    | min heap                     | Merge k Lists       |

## 42. Final Cheat Sheet

### Traverse

```

ListNode* curr = head;

while (curr != nullptr) {
    curr = curr->next;
}

```

### Reverse

```

ListNode* prev = nullptr;
ListNode* curr = head;

while (curr != nullptr) {
    ListNode* nextNode = curr->next;
    curr->next = prev;
    prev = curr;
    curr = nextNode;
}

return prev;

```

## Dummy Node

```
ListNode dummy(0);
dummy.next = head;

return dummy.next;
```

## Merge Two Lists

```
ListNode dummy(0);
ListNode* tail = &dummy;

while (a != nullptr && b != nullptr) {
    if (a->val <= b->val) {
        tail->next = a;
        a = a->next;
    } else {
        tail->next = b;
        b = b->next;
    }

    tail = tail->next;
}

tail->next = a != nullptr ? a : b;
```

## Fast Slow

```
ListNode* slow = head;
ListNode* fast = head;

while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
}
```

## Remove Nth From End

```
ListNode dummy(0);
dummy.next = head;

ListNode* fast = &dummy;
ListNode* slow = &dummy;

for (int i = 0; i < n; i++) {
    fast = fast->next;
```

```
}

while (fast->next != nullptr) {
    fast = fast->next;
    slow = slow->next;
}

slow->next = slow->next->next;
```

## 43. What to Master Before Chapter 17

Before moving to Fast and Slow Pointers, make sure you can:

- Traverse a linked list
- Reverse a linked list
- Save nextNode before rewiring
- Use dummy nodes
- Delete nodes safely
- Merge two sorted lists
- Use tail pointer
- Use fast and slow pointers for middle
- Detect cycles
- Remove nth node from end
- Split a linked list
- Reverse second half of a list
- Reconnect nodes without losing them
- Avoid null pointer dereference
- Explain pointer changes clearly

## 44. Core Takeaway

Linked list problems are pointer surgery.

The most important rule is:

Never change a pointer unless you already saved what you still need.

Common linked list tools:

- curr for traversal
- prev for previous node
- nextNode to avoid losing rest of list
- dummy node for head changes
- fast/slow pointers for middle and cycle
- tail pointer for building lists

If arrays are about indices, linked lists are about connections.

Mastering linked lists means learning to confidently answer:

Which node should point to which node next?

---